

• Medium • #40 • Combination Sum - II

▲ Problem Statement → Given a collection of candidate numbers & a "target", find all unique combinations whose sum equals the target. Each no. can only be used once.

▲ Main Problem Solution → Use Backtracking after sorting the array.

Sorting helps us:-
• Skip duplicate candidates at the same recursion level.

• Stop early when "candidates[i] > target"

Unlike Combination Sum, here we move to "i+1" after choosing a number because each element can only be used once.

▲ Brute Force Method → Generate every possible subset & check whether its sum equals the target.

• Time → $O(2^n)$

• Space → $O(n)$

▲ Algorithm → 1 → Sort "candidates"

2. Start backtracking from index "0"
3. If "target == 0", store the current combination
4. Loop through candidates from "start"
5. Skip duplicates when " $i > \text{start}$ " & " $\text{candidates}[i] == \text{candidates}[i-1]$ "
6. If the current candidate is greater than the target, stop
7. Add the candidates to the current combination
8. Recursively call with " $i+1$ " because each no. can be used only once
9. Remove the candidate using backtracking.
10. Return all unique combinations.

▲ Important Points →

- Sort first to handle duplicates easily.

- " $i > \text{start}$ " ensures we skip duplicates only at the recursion level.
- Use " $i+1$ " in recursion because each element can be used only once.
- "current, push-back()" → Choose

- "current.back()" → undo choice
- "candidates[i] > target" → stop because the array is sorted.
- Duplicate values can still be used if they occur at different indices. For example, [1, 1, 6] is valid.

▲ Complexity → Time → $O(2^n)$
 Space → $O(n)$